

Visão Geral do Projeto	2
Padrões de Projeto Utilizados	2
1. Facade (Fachada):	2
2. Command (Comando):	2
3. State Management (Gerenciamento de Estado para Undo/Redo):	2
4. Polimorfismo (similar à Strategy):	3
Lógicas de Negócio Principais	3
1. Cálculo de Salário Bruto:	3
2. Agendas de Pagamento (SistemaFolha.deveReceber):	3
3. Descontos (SistemaFolha.calcularDescontos):	4
4. Persistência de Dados:	4
Descrição das Principais Classes	4
• Facade (br.ufal.ic.p2.wepayu)	4
• SistemaFolha (br.ufal.ic.p2.wepayu.models)	4
• Empregado (br.ufal.ic.p2.wepayu.models)	5
• EmpregadoHorista (br.ufal.ic.p2.wepayu.models)	5
• EmpregadoComissionado (br.ufal.ic.p2.wepayu.models)	5
• EmpregadoAssalariado (br.ufal.ic.p2.wepayu.models)	5
• Classes Command (br.ufal.ic.p2.wepayu.commands)	5
• Classes de Exceção (br.ufal.ic.p2.wepayu.Exception)	6

Visão Geral do Projeto

Este projeto é um sistema de folha de pagamento chamado "WePayU". Ele é projetado para ser operado através da classe `Facade`, que serve como um ponto de entrada simplificado para um subsistema mais complexo, centrado na classe `SistemaFolha`. O sistema gerencia o cadastro de empregados, o lançamento de dados (cartões de ponto, vendas, taxas de serviço) e a execução da folha de pagamento (`rodaFolha`).

Uma característica central do projeto é a implementação robusta das funcionalidades de "undo" (desfazer) e "redo" (refazer) para todas as ações que modificam o estado do sistema.

Padrões de Projeto Utilizados

O projeto utiliza diversos padrões de projeto clássicos para organizar sua lógica e manter a coesão.

1. Facade (Fachada):

- **Classe:** `br.ufal.ic.p2.wepayu.Facade`.
- **Propósito:** Fornece uma interface única e simplificada para todas as operações do sistema (criar empregado, lançar dados, rodar folha, etc.). Ele esconde a complexidade interna do `SistemaFolha` e é o único ponto de entrada para os testes (como visto em `Main.java`).

2. Command (Comando):

- **Interface:** `br.ufal.ic.p2.wepayu.commands.Command`.
- **Propósito:** Encapsula cada operação que modifica o estado do sistema (como `CriarEmpregadoCommand`, `AlteraEmpregadoCommand`, `LancaCartaoCommand`) em um objeto.
- **Funcionamento:**
 - A interface `Command` define os métodos `execute()` e `undo()`.
 - O `SistemaFolha` atua como "Receiver", contendo a lógica de negócio real que os comandos invocam.
 - A `Facade` atua como "Invoker", recebendo a solicitação, instanciando o comando apropriado e chamando `sistema.executarComando(command)`.
 - `SistemaFolha` gerencia duas pilhas (`undoStack` e `redoStack`) para permitir desfazer e refazer ações.

3. State Management (Gerenciamento de Estado para Undo/Redo):

- **Classe Utilitária:** `br.ufal.ic.p2.wepayu.utilities.Clonador`.

- **Propósito:** Para que o `undo` funcione corretamente, especialmente em operações complexas como `alteraEmpregado` ou `rodaFolha`, o sistema precisa salvar o estado *anterior* à execução do comando.
- **Funcionamento:** A classe `Clonador` usa serialização XML (`XMLEncoder` e `XMLDecoder`) para criar "deep copies" (cópias profundas) de objetos, como o mapa completo de empregados.
- **Exemplo:** O `AlteraEmpregadoCommand` clona o mapa de empregados *antes* de executar a alteração. Se `undo()` for chamado, ele restaura o mapa clonado, revertendo efetivamente a mudança. Outros comandos, como `LancaCartaoCommand`, salvam apenas a lista original de cartões de ponto do empregado afetado.

4. Polimorfismo (similar à Strategy):

- **Classes Base:** `br.ufal.ic.p2.wepayu.models.Empregado`.
- **Classes Derivadas:** `EmpregadoHorista`, `EmpregadoAssalariado`, `EmpregadoComissionado`.
- **Propósito:** A lógica de cálculo de salário é diferente para cada tipo de empregado. O sistema usa polimorfismo para lidar com isso.
- **Funcionamento:** A classe base `Empregado` e suas subclasses implementam métodos específicos de cálculo (como `calculaSalarioBruto`). O `SistemaFolha`, ao rodar a folha, invoca o método apropriado com base no tipo do objeto, sem precisar de `if/else` complexos no loop principal.

Lógicas de Negócio Principais

1. Cálculo de Salário Bruto:

- **Horista (`EmpregadoHorista`):** O salário é $(\text{horasNormais} * \text{salario}) + (\text{horasExtras} * \text{salario} * 1.5)$. Horas extras são contadas como qualquer hora que exceda 8 horas no mesmo dia.
- **Assalariado (`EmpregadoAssalariado`):** Recebe seu salário base (fixo) mensalmente, independentemente das horas.
- **Comissionado (`EmpregadoComissionado`):** Recebe uma porção fixa do salário base $(\text{salario} * 12 / 26)$, indicando um pagamento quinzenal) mais um percentual (`comissao`) sobre o total de vendas (`ResultadoDeVenda`) no período.

2. Agendas de Pagamento (`SistemaFolha.deveReceber`):

- O sistema determina se um empregado deve ser pago em uma data específica:
- **Horista:** Recebe toda sexta-feira.
- **Assalariado:** Recebe no último dia útil do mês.
- **Comissionado:** Recebe a cada duas sextas-feiras (quinzenalmente).

3. Descontos (`SistemaFolha.calcularDescontos`):

- Descontos só são aplicados a empregados `sindicalizado = true`.
- **Taxa Sindical:** Um valor diário (`taxaSindical`) é multiplicado pelo número de dias no período de pagamento (ex: 7 dias para horistas, dias no mês para assalariados).
- **Taxas de Serviço:** O sistema soma todas as `TaxaServico` lançadas para o membro do sindicato dentro do período de pagamento.
- **Total Descontos:** `Taxa Sindical (período) + Soma(Taxas de Serviço no período)`.

4. Persistência de Dados:

- O `SistemaFolha` usa serialização XML para persistir dados.
- **Ao iniciar:** O construtor tenta ler o arquivo "database.xml" usando `XMLDecoder` para carregar o `Map<String, Empregado>`.
- **Ao encerrar:** O método `encerrarSistema()` usa `XMLEncoder` para salvar o mapa atual de empregados no "database.xml".

Descrição das Principais Classes

- **Facade** (`br.ufal.ic.p2.wepayu`)
 - Ponto de entrada do sistema.
 - Mantém uma instância de `SistemaFolha`.
 - Expõe métodos públicos para a interface (ex: `criarEmpregado`, `lancaCartao`, `rodaFolha`).
 - Instancia os objetos `Command` apropriados e os passa para o `SistemaFolha` para execução, gerenciando as chamadas de `undo()` e `redo()`.
- **SistemaFolha** (`br.ufal.ic.p2.wepayu.models`)
 - É o "cérebro" da aplicação.
 - **Atributos Principais:** `Map<String, Empregado> empregados`, `Deque<Command> undoStack`, `Deque<Command> redoStack`.
 - **Métodos Principais:**
 - `executarComando(Command)`: Executa um comando, o adiciona à pilha de undo e limpa a pilha de redo.
 - `undo()` / `redo()`: Move comandos entre as pilhas, chamando `command.undo()` ou `command.execute()`.
 - `criarEmpregado(...)`: Valida os dados e adiciona um novo empregado ao mapa.
 - `alteraEmpregado(...)`: Lógica complexa para alterar qualquer atributo de um empregado, incluindo tipo, salário, sindicalização, etc.

- `lancaCartao(..) / lancaVenda(...)` / `lancaTaxaServico(...)`: Delega a adição desses dados ao objeto `Empregado` correspondente.
 - `rodaFolha(data, saida)`: Método principal do payroll. Itera sobre todos os empregados, verifica se `deveReceber()`, calcula salário bruto e descontos, e escreve um relatório formatado em um arquivo de saída.
 - `encerrarSistema()` / Construtor: Gerencia a persistência de dados em XML.
- **Empregado** (`br.ufal.ic.p2.wepayu.models`)
 - Classe base abstrata (embora não marcada como `abstract`) para todos os tipos de empregados.
 - **Atributos:** `nome`, `endereco`, `tipo`, `salario`, `sindicalizado`, `idSindicato`, `taxaSindical`, `metodoPagamento`, `banco`, `agencia`, `contaCorrente`, `ultimoPagamento`.
 - **Coleções:** `ArrayList<TaxaServico> taxasServico`.
- **EmpregadoHorista** (`br.ufal.ic.p2.wepayu.models`)
 - Herda de `Empregado`.
 - **Coleções:** `ArrayList<CartaoPonto> cartoesDePonto`.
 - **Métodos Principais:** `lancaCartao(data, horas)` (adiciona um novo cartão), `getHorasNormaisTrabalhadas(inicio, fim)` (calcula horas normais no período), `getHorasExtrasTrabalhadas(inicio, fim)` (calcula horas extras no período), `calculaSalarioBruto(dataFinal)` (implementa a lógica de pagamento por hora).
- **EmpregadoComissionado** (`br.ufal.ic.p2.wepayu.models`)
 - Herda de `Empregado`.
 - **Atributos:** `comissao` (percentual).
 - **Coleções:** `ArrayList<ResultadoDeVenda> vendas`.
 - **Métodos Principais:** `lancaVenda(valor, data)` (adiciona uma nova venda), `getVendas(inicio, fim)` (soma as vendas no período), `calculaSalarioBruto(dataFinal)` (implementa a lógica de salário fixo quinzenal + comissão).
- **EmpregadoAssalariado** (`br.ufal.ic.p2.wepayu.models`)
 - Herda de `Empregado`.
 - É a classe mais simples, representando um empregado com salário fixo mensal.
- **Classes Command** (`br.ufal.ic.p2.wepayu.commands`)
 - Conjunto de classes que implementam a interface `Command`.
 - **Exemplos:**
 - `CriarEmpregadoCommand`: Salva o `idCriado`. O `undo()` chama `sistema.removerEmpregado(idCriado)`.

- `RemoverEmpregadoCommand`: Salva o objeto `Empregado` que foi removido. O `undo()` o adiciona de volta ao mapa.
- `AlterarEmpregadoCommand`: Salva um clone *profundo* de *todo* o mapa `empregados` antes da alteração. O `undo()` restaura esse mapa.
- `RodaFolhaCommand`: Similar ao `AlterarEmpregadoCommand`, salva um clone do mapa de empregados antes de rodar a folha, pois `rodaFolha` atualiza a data do `ultimoPagamento` dos empregados.
- Classes de Exceção (`br.ufal.ic.p2.wepayu.Exception`)
 - O projeto define um conjunto extenso de exceções customizadas (ex: `EmpregadoNaoExisteException`, `IdentificacaoEmpregadoNulaException`, `SalarioDeveSerNaoNegativoException`, `DataInvalidaException`, `TipoInvalidoException`) para garantir a validação de dados e a integridade das regras de negócio.